Kotlin Backend Developer.Professional

REC

Проверить, идет ли запись

Меня хорошо видно && слышно?

Создаем интерфейс с помощью OpenAPI

Правила вебинара

Активно
участвуем

Off-topic обсуждаем в
OTUS-Kotlin-2025-05

Задаем вопрос
в чат или голосом

Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения

Индивидуально

Время, необходимое
на активность

Пишем в чат

Говорим голосом

Документ

Ответьте себе или
задайте вопрос

План полета

- Общие принципы API
- OpenAPI
- Кодогенерация
- Рефлексия

О курсе

Модуль 1: Введение в Kotlin	Введение	Базовые элементы	Функц. элементы	ООП		
Модуль 2: Расширенные возможности Kotlin	DSL	Корутины	Sequence, Flow	Multi-platform	Interop	Gradle
Модуль 3: Подготовка к разработке	Маркетинг аналитика	Security	Архитек-тура	DevOps		
Модуль 4: Разработка API	OpenAPI	Модели и маперы	Testing	Monitoring		
Модуль 5: Фреймворки и асинхронный транспорт	Spring	Ktor	WebSocket	RabbitMQ	Kafka	Serverless
Модуль 6: Бизнес-логика проекта	Основы БЛ	Модуль БЛ	Валидация			
Модуль 7: Работа с базами данных	Работа с БД	Репози-торий	Конкурен-ция	Postgres	Cassandra	Gremlin
Модуль 8: Развитие бизнес-логики проекта	Машина состояний	Auth	Основы ML	ML в бэкенде		
Модуль 9: Проектная работа	Обзор экосист.	Консуль-тация	Защита			

Команда курса

Окатов Сергей
автор программы и руководитель курса

- Кандидат физико-математических наук,
- 15 лет научной работы в университетах России, Европы, США;
- Руководитель отдела разработки БФТ.ЦР;
- Работа в стартапах и энтерпрайзе, BigData, Backend, Frontend.

Кирилл Крылов
преподаватель

- Kotlin/ML Разработчик в JetBrains;
- Веду курсы Android-разработки в Samsung;
- Full-stack разработчик мобильных приложений на Kotlin/TS;
- Руководитель курса Kotlin Basic.

Земенков Леонид
преподватель

- Kotlin разработчик в Ennabl, Inc.;
- 4 года пишу на Java;
- За 4 года успел поработать в Сбербанке, Yandex и AliExpress.

Непомнящий Евгений
преподаватель

- 15 лет программировал контроллеры на C++; и руководил отделом разработки
- 3 года пишу на Java;
- Последнее время пишу микросервисы на Java в M-Tex.

Таношкин Александр
преподаватель

- Инженер-программист в крупном классифайде недвижимости;
- Работал с Java, TS и другими языками;
- На Kotlin с 2019 г.

Порхун Анастасия
преподаватель

- 10 лет в индустрии
- Опыт работы в телеком- и аутсорскомпаниях, с гос. структурами;
- Проектирование и разработка ПО;
- Выстраивание процессов CI/CD, исследования в сфере ИБ.

Миних Владимир
преподаватель

- 1991 г. – моя первая программа (ZX Spectrum, Basic);
- 1998 г. – Delphi + первый Java опыт;
- 15+ лет – инженер АСУ ТП. Разработка ПО для ПЛК;
- с 2014 г. – Java разработчик, с 2018 г. – опыт с Kotlin.

Никольский Николай
преподаватель

- Java/Kotlin разработчик в ОТП банке.
- Работа с крупными государственными и банковскими проектами.
- Участник opensource

Цели вебинара

После занятия вы сможете

1. Использовать OpenAPI для создания транспортных моделей
2. Генерировать модели для различных фреймворков
3. Создать транспортные модели для учебного проекта

Смысл

Зачем вам это уметь

1. Для создания многоязычных микросервисных проектов
2. Для генерации документации
3. Для совместной работы с аналитиками
4. Для разработки в микросервисной экосистеме

Слайд с домашним заданием

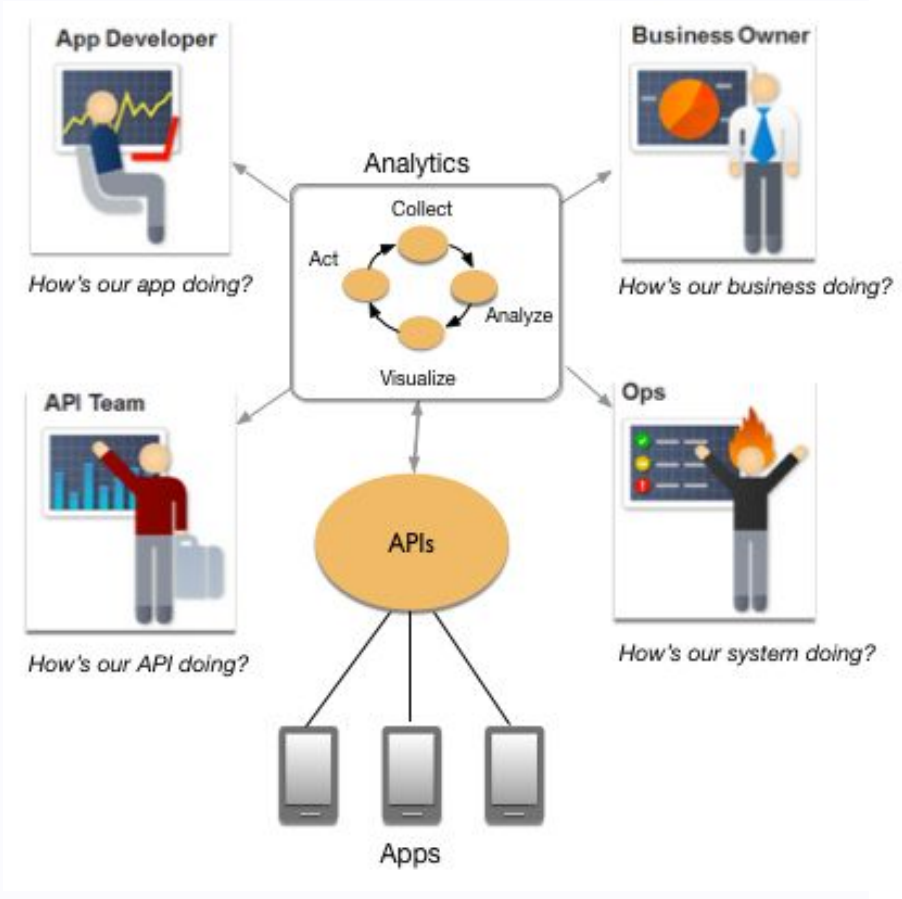
1. Сформировать транспортные модели для вашего выпускного проекта
2. Сделать тест на сериализацию/десериализацию
3. Реализовать внутренние модели
4. Обеспечить маппинг между транспортными и внутренними моделями

Сроки выполнения: **19.08.2025**

Об API

Как формируется API

Кто занимается проектированием API в вашей команде?



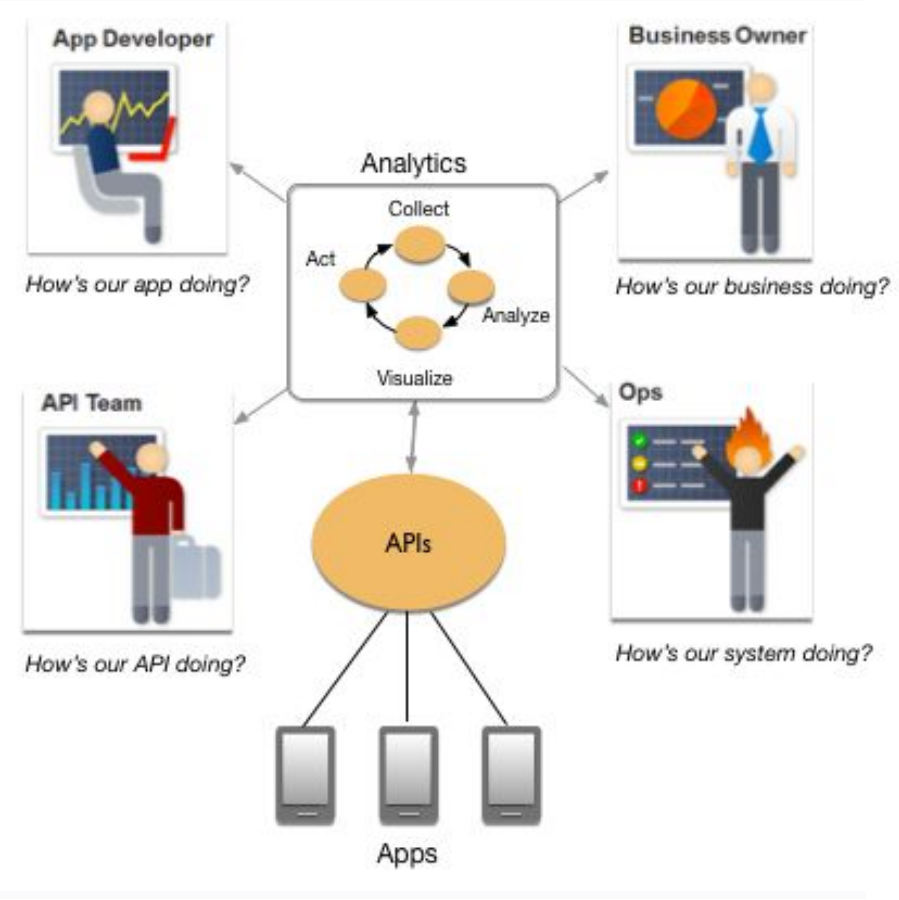
Говорим голосом / Пишем в чат

1 мин

Как формируется API

Базовые принципы:

1. Сбор требований:
   a. Сис.аналитик ← клиенты
   b. Сис.аналитик ← биз.аналитики
   c. Сис.аналитик ← фронтенд
   d. Сис.аналитик ← тестировщик
   e. Сис.аналитик ← бэкенд
   f. Сис.аналитик ← архитектор
2. Сведение и документирование
3. Разработка архитектуры
4. Интеграционные спецификации, т.е. контракт между компонентами



Способы представления спецификации API

Представление спецификации:

1. Текстовые описания
2. Текстовые таблицы
3. Примеры использования
4. Схемы и диаграммы
5. Кодогенерируемые форматы: OpenAPI, ProtoBuf

Яндекс

Найти

Быстрый старт
Условия использования API Яндекс.Карт
Руководство разработчика
Справочник JavaScript API
Примеры в песочнице
Внешние модули
Версии JavaScript API
Руководство по переходу на JSAPI 2.1
Теоретические сведения
Часто задаваемые вопросы
Техническая поддержка

Шаг 4. Создайте карту

В JavaScript-коде создайте экземпляр карты. Конструктору нужно передать:

- идентификатор HTML-контейнера;
- центр карты;
- коэффициент масштабирования.

Пример:

```
<script type="text/javascript">
    // Функция ymaps.ready() будет вызвана, когда
    // загрузятся все компоненты API, а также когда будет готово DOM-дерево.
    ymaps.ready(init);
    function init(){
        // Создание карты.
        var myMap = new ymaps.Map("map", {
            // Координаты центра карты.
            // Порядок по умолчанию: «широта, долгота».
            // Чтобы не определять координаты центра карты вручную,
            // воспользуйтесь инструментом Определение координат.
            center: [55.76, 37.64],
            // Уровень масштабирования. Допустимые значения:
            // от 0 (весь мир) до 19.
            zoom: 7
        });
    }
</script>
```

Внимание. Создавать карту следует после того, как веб-страница загрузится целиком и будет создан контейнер с id. Это нужно, чтобы к нему можно обращаться по `id`. Чтобы

IDL и OpenAPI

IDL – interface definition language

Очень много IDLs

Некоторые примеры:

1. OpenAPI
2. AsyncAPI
3. gRPC
4. GraphQLSchema
5. JsonScema
6. AvroSchema

https://en.wikipedia.org/wiki/Interface_description_language

Examples [edit]

- AIDL: Java-based, for Android; supports local and remote procedure calls, can be accessed from native applications by calling through Java Native Interface (JNI)
- Apache Thrift: from Apache, originally developed by Facebook
- Avro IDL: for the Apache Avro system
- ASN.1
- Cap'n Proto: created by its former maintainer, avoids some of the perceived shortcomings of Protocol Buffers.
- Concise Data Definition Language (CDDL, RFC 8610):[1] A Notation for CBOR and JSON data structures
- CortoScript: Describe data and/or interfaces for systems that require Semantic interoperability
- Etch: Cisco's Etch Cross-platform Service Description Language
- Extensible Data Notation (EDN): Clojure data format, similar to JSON
- FlatBuffers: Serialization format from Google supporting zero-copy deserialization
- Franca IDL: the open-source Franca interface definition language
- FIDL: Interface description language for the Fuchsia Operating System designed for writing app components in C, C++, Dart, Go and Rust.[2]
- IDL specification language: the original Interface Description Language
- IPL: Imandra Protocol Language
- JSON Web-Service Protocol (JSON-WSP)
- Lightweight Imaging Device Interface Language
- Microsoft Interface Definition Language (MIDL): the Microsoft extension of OMG IDL to add support for Component Object Model (COM) and Distributed Component Object Model (DCOM)
- OMG IDL: standardized by Object Management Group, used in CORBA (for DCE/RPC services) and DDS (for data modeling), also selected by the W3C for exposing the DOM of XML, HTML, and CSS documents
- OpenAPI Specification: a standard for Web APIs, used by Swagger and other technologies.

OpenAPI

Общие сведения:

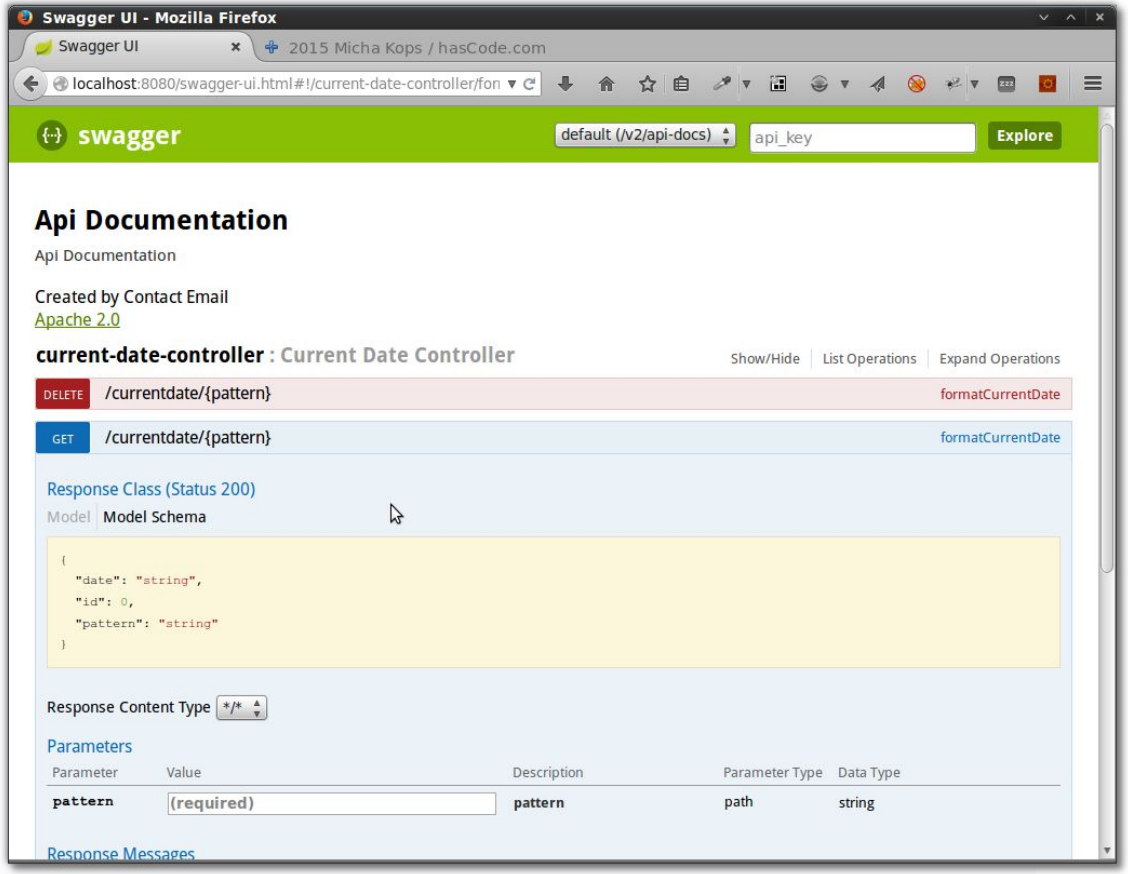
1. Начало разработки в Swagger 2010 by Tony Tam
2. 2011-08-10 First release of the Swagger Specification
3. 2016-01-01, Swagger specification -> OpenAPI Specification (OAS)
4. 2017-07-26 OpenAPI v3
5. Текущая версия 3.1.0

https://spec.openapis.org/oas/latest.html
Инструменты: https://openapi.tools/

OpenAPI - форматы работы

Форматы работы:

1. Подключаем модуль к Spring или Ktor, автогенерируем спеку
   a. Формирует программист
   b. Отдается нечитаемый JSON
   c. Страничка с документацией и формами

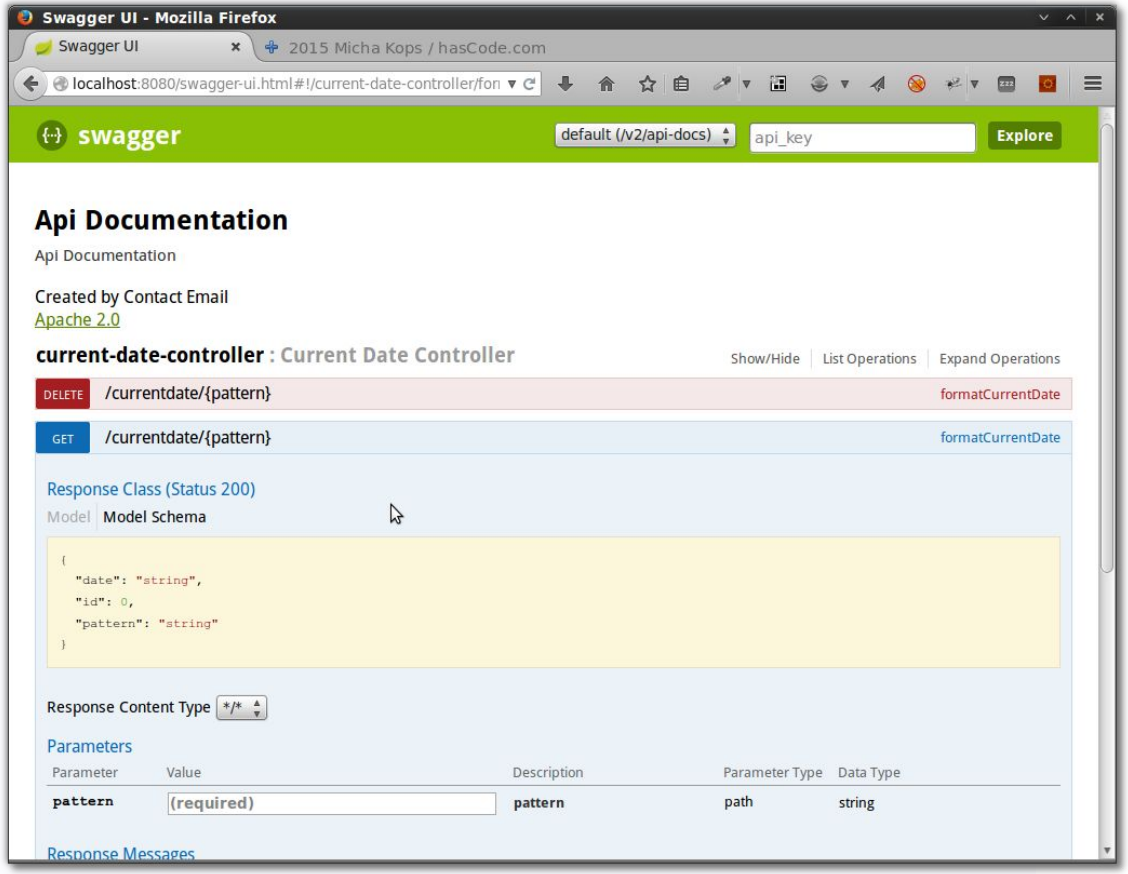


OpenAPI - форматы работы

Форматы работы:

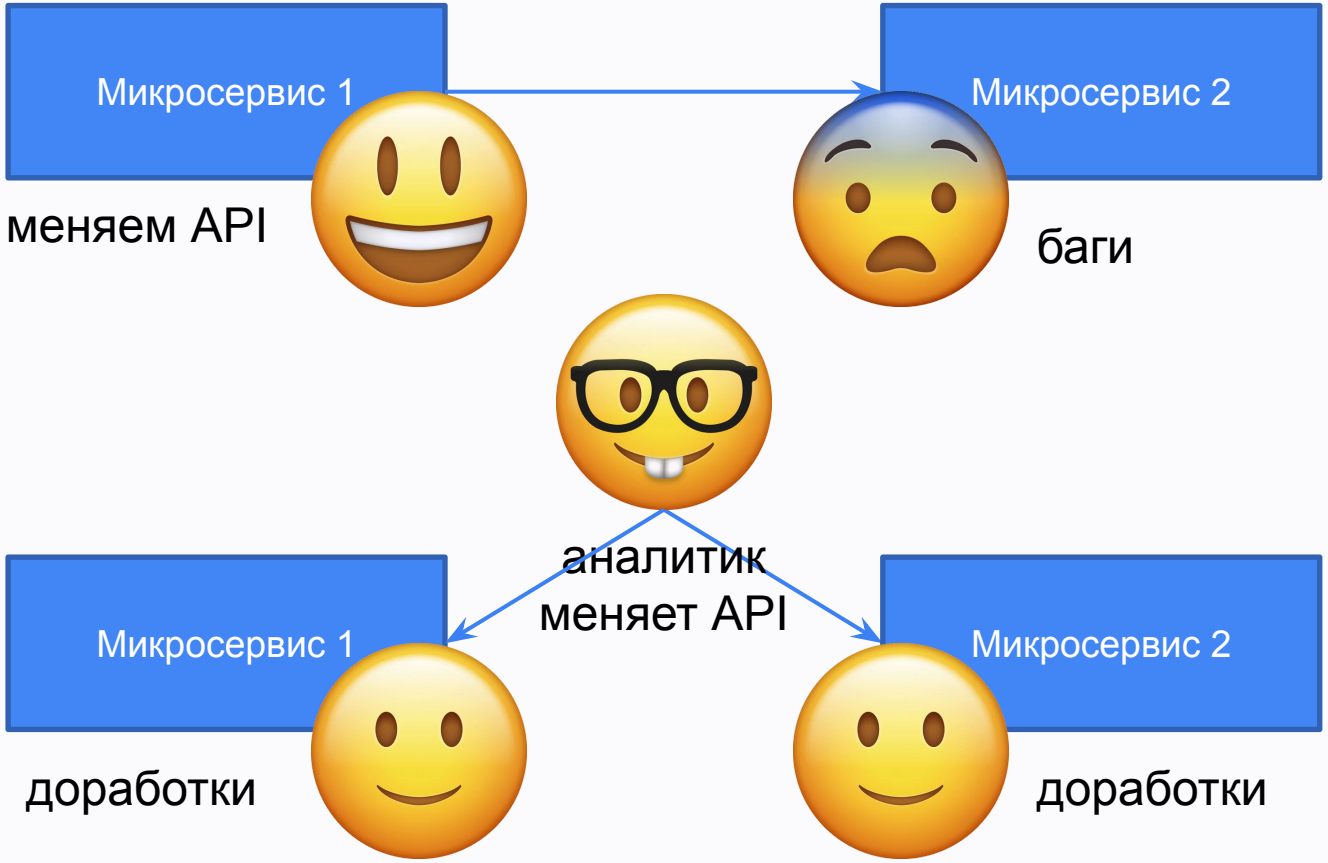
1. ~~Подключаем модуль к Spring или Ktor, автогенерируем спеку~~
   a. ~~Формирует программист~~
   b. ~~Отдается нечитаемый JSON~~
   c. ~~Страничка с документацией и формами~~
2. Пишем спеку и генерируем код для различных платформ
   a. Разрабатывает аналитик
   b. Легко читаемый Yaml
   c. Генерация для всех необходимых платформ одновременно
   d. Возможность подключить UI

https://github.com/papsign/Ktor-OpenAPI-Generator



OpenAPI – в микросервисной экосистеме

Форматы работы:
В МСА синхронизация между командами на уровне API.

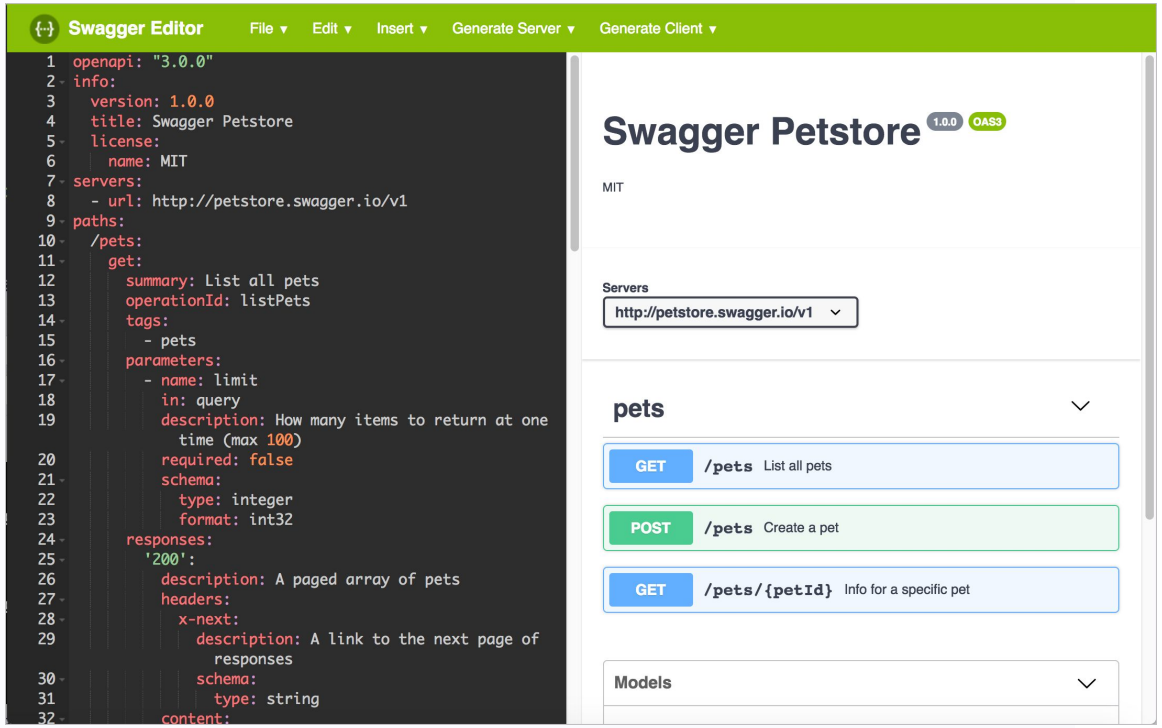


Создание спецификации

Инструменты:

1. Онлайн редактор editor.swagger.io
2. Плагины в Intellij Idea

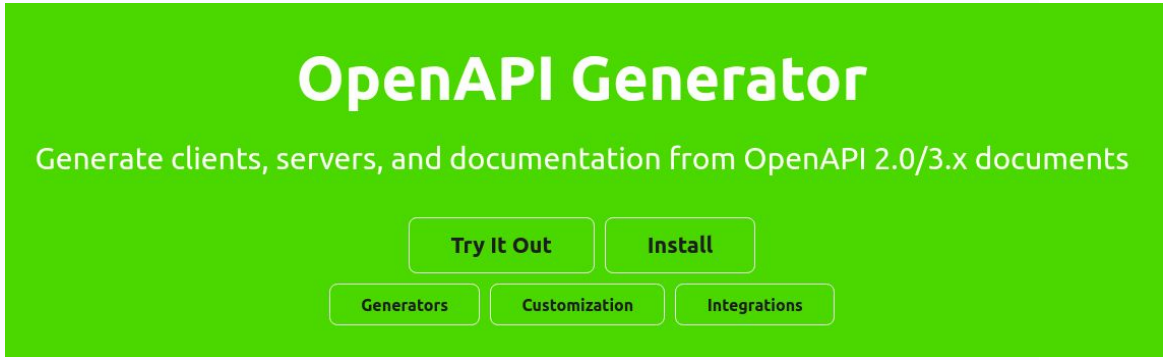
https://editor.swagger.io/



OpenAPI генераторы

Типы генераторов:

1. CLIENT generators
2. SERVER generators
3. DOCUMENTATION generators
4. SCHEMA generators



https://openapi-generator.tech/docs/generators/

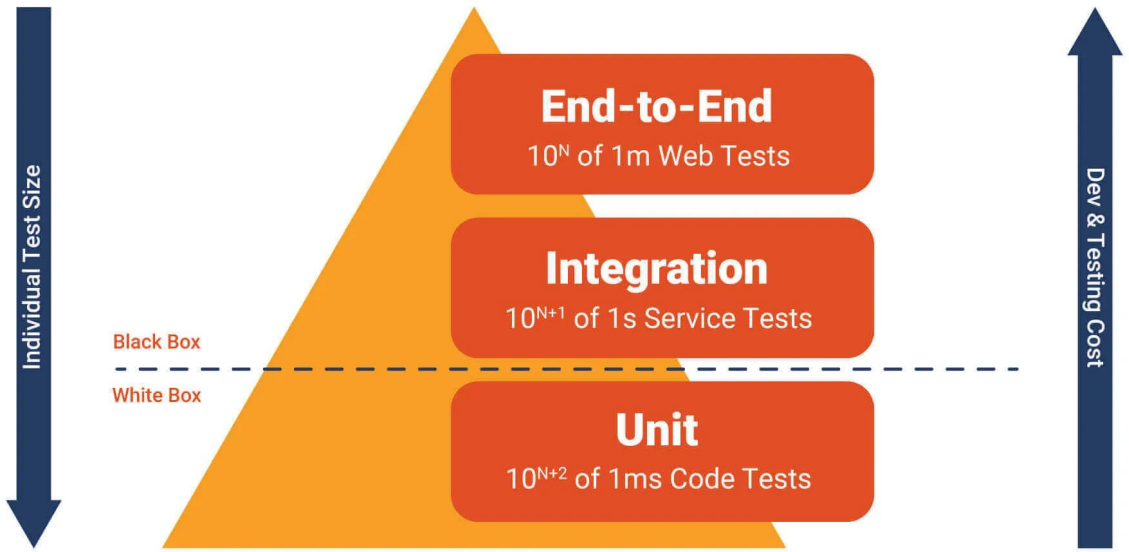
Вопросы?

Разработка API

Дополнение к архитектуре

Требования:

1. Различные уровни тестирования:
   a. модульное
   b. интеграционное
   c. сквозное
2. Инфраструктура для генерации метрик
3. Специальный режим тестирования в приложении:
   a. объект debug в запросах и ответах
4. Каждый запрос содержит идентификатор для отслеживания всего пути его через систему
   a. По стандарту в заголовках HTTP



Особенности API: RESTful и POSTful

RESTful -> POSTful:

1. Следует разделять User API от Backend API:
   a. User API для удобства пользователя
   b. Backend API для фронтенда (SPA, Mobile)
2. Не используем RESTful - устаревшее соглашение, конфликтующее с SPA
3. Не используем GET - спецификацией запрещена передача BODY
4. Не используем параметры строки запроса - усложненный парсинг, большие ограничения
5. Сам REST устарел, ему на смену приходит HTTP2, 3
6. Готовим совместимость с асинхронными транспортными средами

https://example.com/endpoint?param1&param=2
User +
SPA -

POST https://example.com/endpoint
{“param1”:”1”,”param2”:”2”}

User -
SPA +

Особенности API

OpenAPI:

1. Не использует наследование: allOf создает новый объект без связи с исходными
2. Один класс, сгенерированный в разных генераторах, не совместим друг с другом
3. Request/Response body упаковываем как объекты

```
{
  "id": "1",
  "name": "Иван"
}
```

```
{
  "request_user": {
    "id": "1",
    "name": "Иван"
  },
  "response_user": {
    "age": 29,
    "status": "married"
  }
}
```

Особенности API

Kotlin generators:

1. Client
   https://openapi-generator.tech/docs/generators/kotlin
2. Server (ktor)
   https://openapi-generator.tech/docs/generators/kotlin-serv
3. Spring
   https://openapi-generator.tech/docs/generators/kotlin-spri
4. Vertx
   https://openapi-generator.tech/docs/generators/kotlin-vert
5. Multiplatform только через Client

LIVE

Вопросы

1. Кто создает спецификацию API?
2. Как происходит разработка API?
3. Как генерируется код в OpenAPI?

5 мин

Тезисы

Подведем итоги

1. Спеку создает системный аналитик, согласует со всеми, закрепляет контрактом
2. Самый эффективный формат спеки - кодо- и документо-генераторы: OpenAPI, ProtoBuf
3. OpenAPI: на базе кода, на базе спеки
4. editor.swagger.io, Intellij Idea
5. Генерация кода из спеки с помощью плагина Gradle

Слайд с домашним заданием

1. Сформировать транспортные модели для вашего выпускного проекта
2. Сделать тест на сериализацию/десериализацию
3. Реализовать внутренние модели
4. Обеспечить мапинг между транспортными и внутренними моделями
5.

Сроки выполнения: **19.08.2025**

Рефлексия

Цели вебинара

проверка достижения

1. Использовать OpenAPI для создания транспортных моделей
2. Генерировать модели для различных фреймворков
3. Создать транспортные модели для учебного проекта

Рефлексия

С какими основными мыслями и инсайтами уходите с вебинара?

Достигли ли вы цели вебинара?

Следующий вебинар

01.08.2025

Модуль 4: Разработка API

Создаем внутренние модели и маперы

- Ссылка на вебинар будет в ЛК за 15 минут
- Материалы к занятию в ЛК — можно изучать
- Обязательный материал обозначен красной лентой

Список материалов для изучения

- Kotlin: https://kotlinlang.org/
- Gradle: https://gradle.org/
- Учебный проект: https://github.com/otuskotlin/202312-ok-marketplace
- OpenAPI спецификация: https://spec.openapis.org/oas/latest.html
- Генератор Ktor: https://github.com/papsign/Ktor-OpenAPI-Generator
- OpenAPI Generator: https://openapi-generator.tech/docs/generators/
- Editor: https://editor.swagger.io/
- Tools: https://openapi.tools/
- PostFul: https://medium.com/personal-capital-tech-blog/why-we-write-post-only-apis-f15108fb6558

Заполните, пожалуйста, опрос о занятии по ссылке в чате

Спасибо за внимание!
Приходите на следующие вебинары